The Journal of Open Source Software

D-FINE-seg: a PyTorch framework for real-time object detection, instance segmentation, and semantic segmentation with multi-backend deployment

Argo Saakyan [1¶]

1 Agnify.ai, Spain ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 18 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Object detection locates and classifies objects in an image; instance and semantic segmentation go further and outline each object or label every pixel - capabilities needed in fields from drone-based monitoring to document analysis. D-FINE-seg is a pip-installable Python framework that takes a computer vision project from raw annotations to a deployed inference engine. It trains and runs real-time models for three tasks - object detection, instance segmentation, and semantic segmentation - switched by a single configuration flag, in five model sizes with pretrained weights for each task (COCO (Lin et al., 2014) for detection and instance segmentation, Cityscapes (Cordts et al., 2016) for semantic segmentation - those weights inherit Cityscapes' non-commercial terms). It exports to ONNX, TensorRT, OpenVINO, and CoreML, with an automated parity check of each exported backend against the PyTorch reference, and runs inference through one Python API on GPU and CPU.

Statement of need

D-FINE-seg closes the gap between a strong detection architecture and an applied segmentation project. Upstream D-FINE (Peng et al., 2025) ships detection only: open issues on its tracker request instance segmentation and segmentation-style polygon outputs (#149, #174), and there are no segmentation heads or mask-aware losses. Building segmentation on this model family means writing heads, losses, and deployment glue from scratch.

Applied vision projects run a tight loop: annotate, train, inspect errors, fix annotations, retrain, export, deploy. D-FINE-seg ships this loop as one toolchain: training with debugging outputs (augmented batches, validation overlays of predictions against ground truth), a check-errors command that isolates model errors for annotation review, and export with parity checks. Four-channel input (RGB plus thermal (Liu et al., 2022), depth, or near-infrared) is a native code path for multi-modal sensors. The loop reflects the author's production workflow. The intended audience is applied computer-vision researchers who need real-time detection or segmentation in their own domain - drone imagery (Zhu et al., 2022), environmental monitoring (Proença & Simões, 2020), multi-modal RGB+thermal sensing - and practitioners shipping vision models to server and edge hardware.

State of the field

Real-time DETR-family detectors - RT-DETR (Zhao et al., 2024) and D-FINE (Peng et al., 2025), building on DETR (Carion et al., 2020) - compete with YOLO-style models, while strong segmentation baselines such as Mask2Former and SegFormer live in research codebases (Cheng et al., 2022; E. Xie et al., 2021). Among deployable alternatives: MMDetection (Chen

DRAFT

et al., 2019) and Detectron2 (Wu et al., 2019) are research-first toolboxes where deployment is secondary; Ultralytics (Jocher et al., 2026) ships YOLO26 with a semantic-segmentation variant and wide backend export, under an AGPL-3.0 licence; RF-DETR (Robinson et al., 2026) offers a DETR-family detector with instance-segmentation and keypoint variants, but no semantic segmentation. D-FINE-seg differs in the combination of: one D-FINE (DETR-family) architecture across all three tasks; automated cross-backend parity checks; the error-review loop for annotation iteration; Apache-2.0 licensing; and native four-channel input with pretrained-weight adaptation.

D-FINE-seg is a standalone framework rather than a fork of the upstream repository: upstream fine-tuning means cloning the repo and hand-editing its configs, while here the workflow is `pip install dfine-seg` plus a `dfine` CLI (`init`, `split`, `train`, `export`, `bench`) with a fuller training and debugging stack, and segmentation on top. The detection core reuses the D-FINE authors' Apache-2.0 model code with small, weight-compatible modifications; the framework around it - data handling, training loop, export pipeline, inference backends, CLI - is new code. That scope exceeds what a reasonable upstream contribution could carry - it was a framework rewrite around the model, not a head to graft on - so it lives as a separate project instead.

Software design

One codebase serves three tasks through a single task flag. The instance segmentation head follows the Mask DINO paradigm (Li et al., 2023): per-query mask embeddings from a small MLP are dotted with shared multi-scale mask features. Training is mask-aware (box-cropped BCE and Dice losses, mask costs in the Hungarian matcher, mask supervision inside contrastive denoising (H. Zhang et al., 2023)); detection models carry no mask compute - the mask head runs only in the segmentation tasks. The semantic segmentation head reuses the same mask fuser on full-frame features, adds a small convolutional neck and a 1x1 classifier - no queries, no NMS - and exports as a fused-argmax graph with a single integer label-map output.

Training prioritizes practical iteration: Muon/Adan optimization (Jordan et al., 2024; X. Xie et al., 2024), multi-GPU mixed-precision training with automatic batch-size probing, knowledge distillation from a frozen teacher, mosaic and Albumentations augmentation (Buslaev et al., 2020), and Weights & Biases tracking.

Deployment is a first-class subsystem: five backends (PyTorch (Paszke et al., 2019), ONNX Runtime (ONNX contributors, 2017), TensorRT (NVIDIA, 2016), OpenVINO (Intel, 2018), CoreML (Apple, 2017)), with `load_model` recovering size, task, and class count from the checkpoint itself. Exports cover the four graph formats, with INT8 variants for TensorRT and OpenVINO (detection and instance segmentation) and CoreML; a parity self-check then compares each exported backend against the PyTorch reference on a shared input - top-K score agreement for detection and instance segmentation, per-pixel label agreement for semantic segmentation - and writes it next to the weights. A small C++/CUDA core (`libdfine.so`, TensorRT inference with fused pre- and postprocessing kernels, plus an NVDEC->NVENC video pipeline) ships the same three-task inference outside Python. Two trade-offs are deliberate: the graph-backend wrappers duplicate code so each file can be copied into an application on its own, and the semantic head applies argmax before a nearest-neighbour resize to the original image resolution - upsampling logits first improves mIoU only marginally while substantially increasing TensorRT latency.

Continuous integration runs linting, a fast CPU test suite, install checks, and a predict smoke test; fixture-based regression tests over model outputs run locally via `make test`; and for any change that can affect model accuracy, a scripted regression (`scripts/regression_test.py`) retrains each task on Cityscapes and compares training, export, and benchmark metrics against tracked reference runs with explicit tolerances. A simplified ByteTrack tracker (Y. Zhang et al., 2022), SAM 3 auto-labeling (Carion et al., 2026), and a Gradio demo complete the toolchain.

DRAFT

Research impact statement

D-FINE-seg is in active research use. The companion preprint (Saakyan & Solntsev, 2026) reports detection and instance-segmentation experiments on VisDrone (Zhu et al., 2022) and TACO (Proença & Simões, 2020) produced with the framework; a public 28-model Cityscapes benchmark (Cordts et al., 2016) (github.com/ArgoHA/cityscapes-benchmark) evaluates it against official checkpoints of four other model families, with the largest model reaching 84.84 mIoU at 14.64 ms end-to-end on native-resolution val (83.23 mIoU on the official test server; RTX 5070 Ti, TensorRT FP16, batch 1); outside groups build on it - the University of Canterbury's UC Vision group forked it for vineyard scanning and yield estimation (uc-vision/D-FINE-seg), the author of the kraken text-recognition engine adapted it for historical document layout analysis (dfine_kraken, published on PyPI as the basis for kraken integration), and users train it for text-line segmentation of manuscripts (#36); and 56 of its 57 GitHub issues were opened by outside users. It is also deployed commercially for document processing (Veryfi Inc.), video monitoring and industrial inspection (Agnify.ai). Adoption at the time of writing: roughly 1,500 PyPI downloads, roughly 1,400 all-time downloads of the pretrained weights on Hugging Face (828 in the last 30 days), 217 GitHub stars and 36 forks.

AI usage disclosure

From its first commit in January 2025 through October 2025, D-FINE-seg was written by the author without AI agents; this period produced the detection framework. AI use in that period was limited to inline code completion (GitHub Copilot in VS Code) and occasional questions to ChatGPT (models not recorded). Since November 2025, AI coding agents have been used for parts of the implementation, refactoring, test scaffolding, debugging, and documentation: GitHub Copilot in VS Code (Claude Opus 4.5, 4.6), Claude Code (Claude Opus 4.7, 4.8, 5.0, 5.5; Fable 5.0, 5.1), OpenAI Codex (Astra, Sol 6.1), and Prime Agent (GLM 5.3 Flash). The core codebase, including the instance-segmentation head, was written directly by the author in VS Code with AI assistance in some places; the author designed the architectures, made all design decisions, and reviewed, tested, and validated all AI-assisted code before release.

This manuscript was drafted and revised with AI assistance (Prime Agent, GLM 5.3 Flash; Claude Code with Claude Opus 5.5, which reviewed drafts against the JOSS requirements and suggested revisions) and reviewed, edited, and validated by the author, who is responsible for its content. No generative AI was used in communications with JOSS editors or reviewers.

Acknowledgements

The detection core reuses the D-FINE authors' Apache-2.0 implementation, and the mask head design follows Mask DINO - the author thanks both teams. This work received no external funding.

Conflicts of interest

The author was employed by Veryfi Inc. from 2023 to 2026 and is currently a Principal Computer Vision Engineer at Agnify.ai; D-FINE-seg is deployed in production at both companies (see the Research impact statement). Apart from this employment, the author declares no competing financial, personal, or professional interests.

References

Apple. (2017). *Coremltools*. https://github.com/apple/coremltools.

DRAFT

Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. A. (2020). Albumentations: Fast and flexible image augmentations. *Information*, *11*(2), 125. https://doi.org/10.3390/info11020125

Carion, N., Gustafson, L., Hu, Y.-T., & others. (2026). SAM 3: Segment anything with concepts. *The Fourteenth International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.2511.16719

Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. *European Conference on Computer Vision (ECCV)*, 213–229. https://doi.org/10.1007/978-3-030-58452-8_13

Chen, K., Wang, J., Pang, J., Cao, Y., Xiong, Y., Li, X., Sun, S., Feng, W., Liu, Z., Xu, J., & others. (2019). MMDetection: Open MMLab detection toolbox and benchmark. *arXiv Preprint arXiv:1906.07155*. https://doi.org/10.48550/arXiv.1906.07155

Cheng, B., Misra, I., Schwing, A. G., Kirillov, A., & Girdhar, R. (2022). Masked-attention mask transformer for universal image segmentation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 1280–1289. https://doi.org/10.1109/CVPR52688.2022.00135

Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., & Schiele, B. (2016). The cityscapes dataset for semantic urban scene understanding. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 3213–3223. https://doi.org/10.1109/CVPR.2016.350

Intel. (2018). *OpenVINO toolkit*. https://docs.openvino.ai.

Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F. C., & Kalfaoglu, M. E. (2026). Ultralytics YOLO26: Unified real-time end-to-end vision models. *arXiv Preprint arXiv:2606.03748*. https://doi.org/10.48550/arXiv.2606.03748

Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., & Bernstein, J. (2024). *Muon: An optimizer for hidden layers in neural networks*. https://kellerjordan.github.io/posts/muon/.

Li, F., Zhang, H., Xu, H., Liu, S., Zhang, L., Ni, L. M., & Shum, H.-Y. (2023). Mask DINO: Towards a unified transformer-based framework for object detection and segmentation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 3041–3050. https://doi.org/10.1109/CVPR52729.2023.00297

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. *European Conference on Computer Vision (ECCV)*, 740–755. https://doi.org/10.1007/978-3-319-10602-1_48

Liu, J., Fan, X., Huang, Z., Wu, G., Liu, R., Zhong, W., & Luo, Z. (2022). Target-aware dual adversarial learning and a multi-scenario multi-modality benchmark to fuse infrared and visible for object detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 5792–5801. https://doi.org/10.1109/CVPR52688.2022.00571

NVIDIA. (2016). *TensorRT*. https://developer.nvidia.com/tensorrt.

ONNX contributors. (2017). *ONNX: Open neural network exchange*. https://onnx.ai.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*. https://doi.org/10.48550/arXiv.1912.01703

Peng, Y., Li, H., Wu, P., Zhang, Y., Sun, X., & Wu, F. (2025). D-FINE: Redefine regression task in DETRs as fine-grained distribution refinement. *The Thirteenth International Conference*

DRAFT

on Learning Representations (ICLR). https://doi.org/10.48550/arXiv.2410.13842

Proença, P. F., & Simões, P. F. (2020). TACO: Trash annotations in context for litter detection. *arXiv Preprint arXiv:2003.06975*. https://doi.org/10.48550/arXiv.2003.06975

Robinson, I., Robicheaux, P., Popov, M., Ramanan, D., & Peri, N. (2026). RF-DETR: Neural architecture search for real-time detection transformers. *The Fourteenth International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.2511.09554

Saakyan, A., & Solntsev, D. (2026). D-FINE-seg: Object detection and instance segmentation framework with multi-backend deployment. *arXiv Preprint arXiv:2602.23043*. https://doi.org/10.48550/arXiv.2602.23043

Wu, Y., Kirillov, A., Massa, F., Lo, W.-Y., & Girshick, R. (2019). *Detectron2*. https://github.com/facebookresearch/detectron2.

Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J. M., & Luo, P. (2021). SegFormer: Simple and efficient design for semantic segmentation with transformers. *Advances in Neural Information Processing Systems (NeurIPS)*. https://doi.org/10.48550/arXiv.2105.15203

Xie, X., Zhou, P., Li, H., Lin, Z., & Yan, S. (2024). Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *46*(12), 9508–9520. https://doi.org/10.1109/TPAMI.2024.3423382

Zhang, H., Li, F., Liu, S., Zhang, L., Su, H., Zhu, J., Ni, L. M., & Shum, H.-Y. (2023). DINO: DETR with improved DeNoising anchor boxes for end-to-end object detection. *The Eleventh International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.2203.03605

Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. *European Conference on Computer Vision (ECCV)*, 1–21. https://doi.org/10.1007/978-3-031-20047-2_1

Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., & Chen, J. (2024). DETRs beat YOLOs on real-time object detection. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 16965–16974. https://doi.org/10.1109/CVPR52733.2024.01605

Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., & Ling, H. (2022). Detection and tracking meet drones challenge. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *44*(11), 7380–7399. https://doi.org/10.1109/TPAMI.2021.3119563

DRAFT